Digital Twins for Greenhouses

Project Onboarding & Architecture Guide

October 2026

Welcome!
This document contains the complete technical blueprint for our greenhouse digital twin project. Whether you are working on hardware, data engineering, or machine learning, this guide explains exactly what we have built so far, the tools we will use, and our roadmap to autonomous AI control.

Contents

1 Introduction: What is a Digital Twin? 2

2 Current Status: What We Have Built 2
2.1 Our Hardware Prototype 2
2.2 Why Our Current Code is Not a Twin 2

3 The Architecture: Moving Beyond Pure Python 3
3.1 Hardware Strategy & Data Privacy 3
3.2 The Data Stack: Python vs. Dedicated Tools 3

4 Part 4: The Brain (Models and Machine Learning) 3
4.1 The Physics Model 3
4.2 Machine Learning Tasks 4

5 Part 5: Autonomous Control (Reinforcement Learning) 4
5.1 How we will deploy RL 4

6 Part 6: Our 4-Phase Roadmap 4

7 Conclusion: Common Pitfalls to Avoid 5

1 Introduction: What is a Digital Twin?

Before writing code or wiring sensors, we need to agree on our definitions. In the industry (and heavily referenced in the Wageningen University *AGROS* trials), there is a strict hierarchy of digital representations:

1. **Digital Model:** A static representation. A 3D CAD file or an Excel spreadsheet calculating heat loss is a model. There is no automatic data link to reality.
2. **Digital Shadow:** A dashboard. Sensors automatically send data from the physical greenhouse to a database, and we view it on a screen. Data flows one way. **We are currently building this.**
3. **Digital Twin:** A closed loop. A digital twin runs a physics or AI simulation in sync with the real greenhouse. It uses this simulation to forecast the future (e.g., predicting overheating) and automatically sends **setpoints** back down to the hardware to change the physical state (e.g., opening a vent). Data flows two ways.

Our ultimate goal is a Digital Twin, but we must build the Shadow first.

2 Current Status: What We Have Built

We currently have a highly functional **Rule-Based Edge Controller** written purely in Python.

2.1 Our Hardware Prototype

- **The Brain:** We are using a **Raspberry Pi** acting as our main local server. We are committed to keeping all processing and data storage **strictly local** within the lab—no external cloud dependencies. Data will be saved permanently to an external SSD attached to the Pi.
- **The Microcontroller:** While we originally tested with a raspberry pi 4, we are migrating to an **Arduino** to read the sensors and drive the relays.
- **Sensors (I2C):** We have successfully integrated the **SHT30** (temperature/humidity) and a **CO2 sensor**, running through concurrent Python threads (`sensors.py`).
- **Actuators (GPIO):** We have wired relays for the fan, heater, irrigation, and a grow light on GPIO 6 (`actuators.py`).

2.2 Why Our Current Code is Not a Twin

Right now, our `actuators.py` script contains logic like: *"If Zone 1 Humidity > 70%, turn off the grow light."* This is reactive, rule-based logic. A true digital twin is predictive. It will say: *"Based on the current trajectory and weather, humidity will hit 75% in 30 minutes, so I will actuate the fan now to prevent it."*

Furthermore, our Python prototype saves data to a local SQLite database in the Pi's RAM (`/dev/shm`). If the Pi reboots, we lose our history. To train machine learning models later, we need months of permanent data.

3 The Architecture: Moving Beyond Pure Python

3.1 Hardware Strategy & Data Privacy

Currently, our Raspberry Pi handles both the sensor reading and the logic processing. Moving forward, we plan to transition the edge sensing to **Arduino** microcontrollers (placed directly in the wet greenhouse) to act as pure sensor-readers and relay-drivers. The Raspberry Pi will remain as our central control hub.

Crucially, we will not use external cloud servers. For security, reliability, and cost reasons, all computation will happen locally inside our lab. Data will be stored safely on an external Solid State Drive (SSD) physically connected to our local hub.

3.2 The Data Stack: Python vs. Dedicated Tools

We are currently evaluating whether to build our data pipeline entirely in Python or to use an industry-proven open-source IoT stack. While Python is unmatched for machine learning, dedicated tools are often better for stable infrastructure. Here is a breakdown of the specific steps and the tools built for them:

- **Data Transport (MQTT Broker - Mosquitto):** Instead of Python continuously running loops to ask sensors for data, Arduinos publish data to Mosquitto (a messaging system). It is incredibly fast, lightweight, and guarantees data delivery even if other parts of the system are restarting.
- **Data Routing (Node-RED):** A visual tool that catches MQTT messages, formats them, and routes them to the database. We can use this instead of writing custom Python data-cleaning pipelines that might crash upon receiving a bad sensor reading.
- **Time-Series Storage (InfluxDB):** Unlike our current SQLite setup or saving CSV files in Python, InfluxDB is explicitly designed to handle thousands of timestamped sensor readings per second and store them permanently and efficiently on our local SSD without database locks.
- **Visualization (Grafana):** A professional visualization tool that connects to InfluxDB to create live dashboards and send alerts. Building a live 24/7 dashboard in Python (e.g., with Streamlit) often leads to memory leaks and crashes over time.
- **State Management (Eclipse Ditto):** An open-source framework specifically for digital twins. It safely separates a device's "reported state" (what the Arduino says the humidity is) from its "desired state" (what the AI wants the humidity to be), which is complex and error-prone to code manually in Python.

Regardless of the infrastructure we ultimately choose for data routing, Python will remain our core language for what it does best: the complex physics models and Machine Learning.

4 Part 4: The Brain (Models and Machine Learning)

This is where we return to Python. Once data is flowing into InfluxDB, we need to create the "Twin Core". We will use a hybrid approach:

4.1 The Physics Model

We will use **GreenLight**, an open-source Python platform. It uses balance equations for heat, humidity, and CO_2. However, a generic model is useless. We must **calibrate** it. We will use our historical data to tweak the model's parameters (like how leaky our specific greenhouse walls

are) until the model's output perfectly matches reality. To keep it accurate over time, we use state estimators (like Kalman filters) to constantly correct the virtual model using live sensor readings.

4.2 Machine Learning Tasks

We will integrate Machine Learning (ML) into the twin in several ways:

- **Soft Sensing:** Using math to estimate what we cannot physically measure.
- **Forecasting:** Using algorithms like LSTM or XGBoost to predict the final crop yield based on the climate history.
- **Computer Vision:** Using cameras and convolutional neural networks to detect plant health and dimensions.

5 Part 5: Autonomous Control (Reinforcement Learning)

How do we teach the twin to make optimal decisions? We use **Reinforcement Learning (RL)**.

In RL, an **Agent** (a neural network) interacts with an **Environment** (our greenhouse). The Agent takes an **Action** (e.g., turning on a heater). The Environment changes **State** (temperature goes up), and the Agent receives a **Reward** (e.g., +10 points for growing fruit, −5 points for spending money on electricity).

5.1 How we will deploy RL

Training an RL agent requires millions of trial-and-error attempts. We absolutely cannot train it on the physical greenhouse, or we will kill the plants and destroy the equipment.

1. **Wrap the Simulator:** We take our calibrated GreenLight model and turn it into a fast, simulated environment.
2. **Train with Variation:** We train the agent in simulation using tools like Stable-Baselines3, randomizing the virtual weather so the AI learns robust strategies and doesn't overfit.
3. **Beat the Baseline:** We test the AI in simulation against standard rule-based controllers and expert grower strategies.
4. **Shadow Mode:** Once trained, we feed the live greenhouse data to the AI. It calculates what it *would* do, but does not act. We just watch and compare its recommendations to reality.
5. **Limited Autonomy:** Finally, we allow it to act, but behind a strict safety layer. The twin suggests "setpoints" (target temperatures), but hardcoded limits on the physical hardware ensure it can never do something catastrophic.

6 Part 6: Our 4-Phase Roadmap

We will not build this all at once. We will follow a strict, four-phase plan. Each phase delivers a usable product and clearly maps to our core engineering definitions:

- **Phase 1: The Digital Shadow.** Move our Python sensor code to MQTT. Set up our local SSD storage (using tools like InfluxDB and Grafana, or Python equivalents). Start safely logging data and actuator states. *Status: Data flows automatically up from the physical greenhouse, but humans push the buttons (Manual Down).*

- **Phase 2: Introducing the Digital Model.** Calibrate the GreenLight climate physics model to our greenhouse using our saved SSD data. Ask it to forecast the next day's climate and measure its error against reality. *Status: We have a working predictive model, but it does not control anything.*

- **Phase 3: The Advisory Digital Twin.** Add the crop models and ML soft sensors. Run "what-if" scenarios. The system calculates the best setpoints and recommends actions. *Status: The loop is almost closed, but humans still authorize the actions.*

- **Phase 4: The Autonomous Digital Twin.** First, we implement model-based control. Then, we bring in the RL agent via shadow mode, eventually granting it limited, safe autonomy over the physical actuators. *Status: Data flows automatically up, and AI decisions flow automatically down.*

7 Conclusion: Common Pitfalls to Avoid

As we execute this plan, we must watch out for standard failures:

- **Sensor Drift:** Cheap sensors degrade. We must regularly validate our SHT30 data and recalibrate.

- **Sim-to-Real Gap:** AI will exploit glitches in the simulator. What works in code might fail in the dirt. This is why Shadow Mode is mandatory.

- **Trust and Safety:** If the autonomy is not explainable, or if it crashes the system, the human team will pull the plug. Safety limits must always live on the physical microcontroller (Arduino), never just in the AI software.

"Start with a shadow, earn the twin."